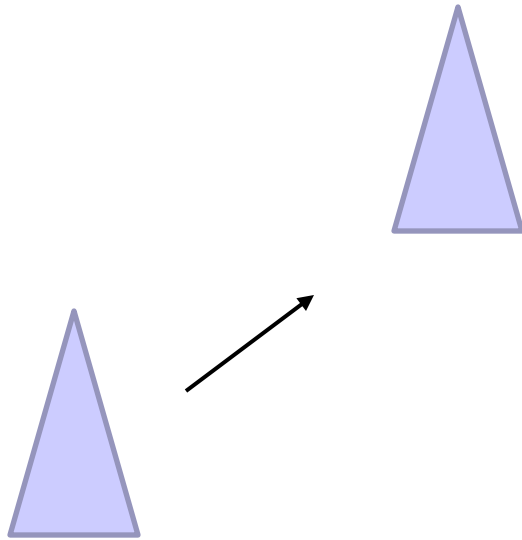


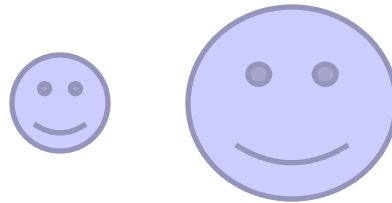
2D Transformations

2D Transformations



Translate

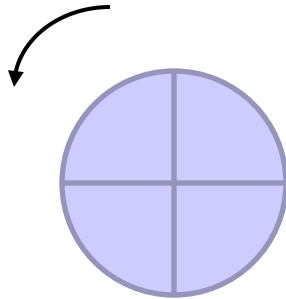
2D Transformations



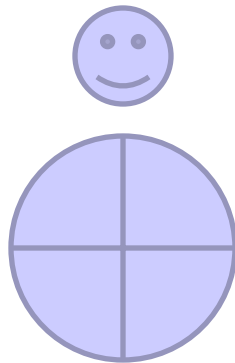
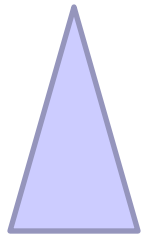
Scale

2D Transformations

Rotate



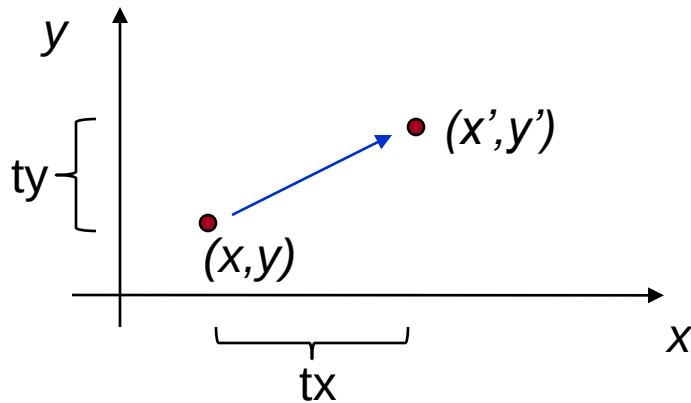
2D Transformations



Translation

$$x' = x + tx$$

$$y' = y + ty$$

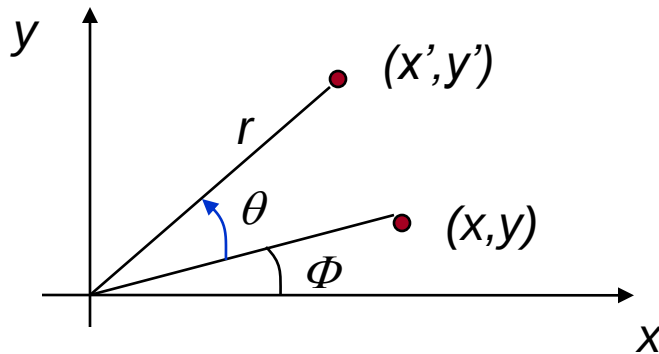


translation vector: $T = \begin{bmatrix} tx \\ ty \end{bmatrix}$

$$P' = \begin{bmatrix} x' \\ y' \end{bmatrix} \quad P = \begin{bmatrix} x \\ y \end{bmatrix}$$

$$P' = P + T$$

Rotation



$$x = r.\cos\Phi$$
$$y = r.\sin\Phi$$

Rotation about the origin

$$x' = r.\cos(\Phi+\theta) = r.\cos\Phi\cos\theta - r.\sin\Phi\sin\theta$$

$$y' = r.\sin(\Phi+\theta) = r.\cos\Phi\sin\theta + r.\sin\Phi\cos\theta$$

$$x' = x.\cos\theta - y.\sin\theta$$

$$y' = x.\sin\theta + y.\cos\theta$$

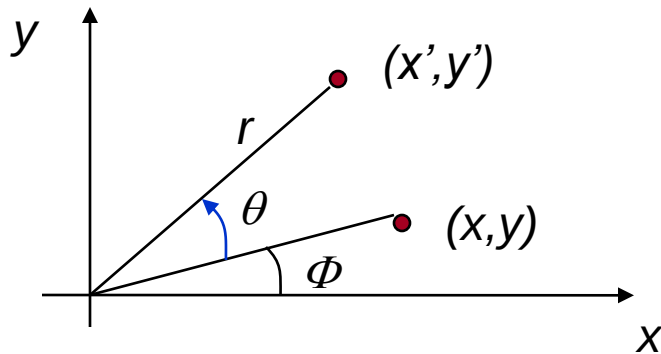
Rotation matrix:

$$P' = \begin{bmatrix} x' \\ y' \end{bmatrix} \quad R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \quad P = \begin{bmatrix} x \\ y \end{bmatrix}$$

$$P' = R . P$$

Rotation

Rotation about the origin

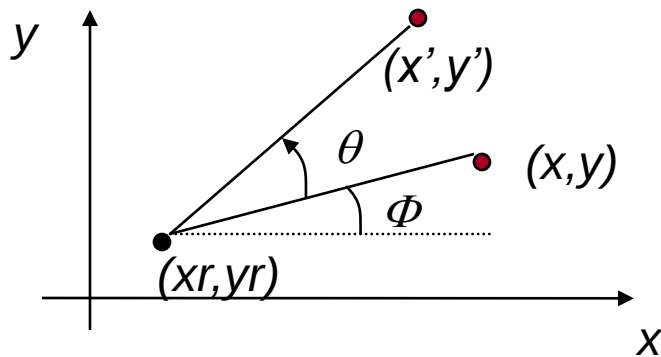


Rotation matrix:

$$R = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

$$P' = R \cdot P$$

Rotation

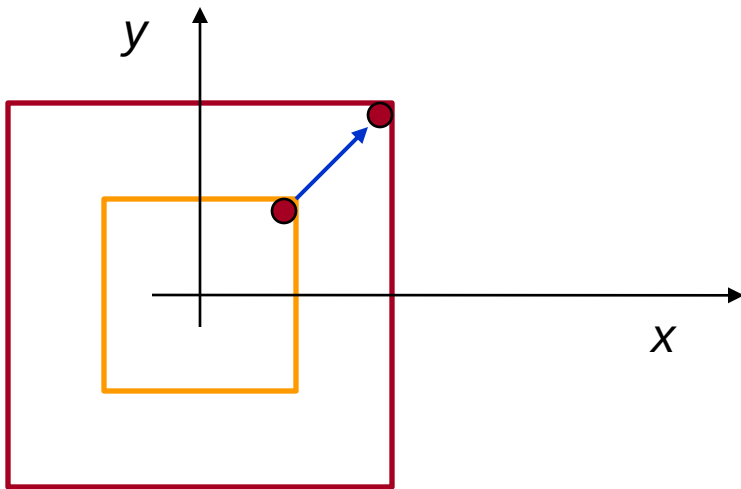


***Rotation about an
arbitrary point (x_r, y_r)***

$$x' = x_r + (x - x_r) \cdot \cos \theta - (y - y_r) \cdot \sin \theta$$

$$y' = y_r + (x - x_r) \cdot \sin \theta + (y - y_r) \cdot \cos \theta$$

Scaling



$$x' = x \cdot sx$$

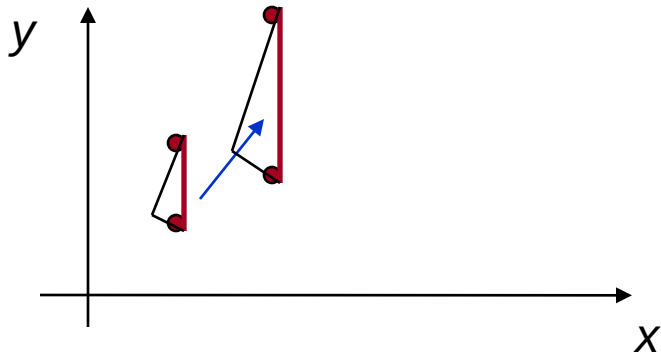
$$y' = y \cdot sy$$

$$\text{Scaling matrix : } S = \begin{pmatrix} sx & 0 \\ 0 & sy \end{pmatrix}$$

$$P' = S \cdot P$$

$$P' = \begin{pmatrix} x' \\ y' \end{pmatrix} \quad S = \begin{pmatrix} sx & 0 \\ 0 & sy \end{pmatrix} \quad P = \begin{pmatrix} x \\ y \end{pmatrix}$$

Scaling



$$x' = x \cdot sx$$

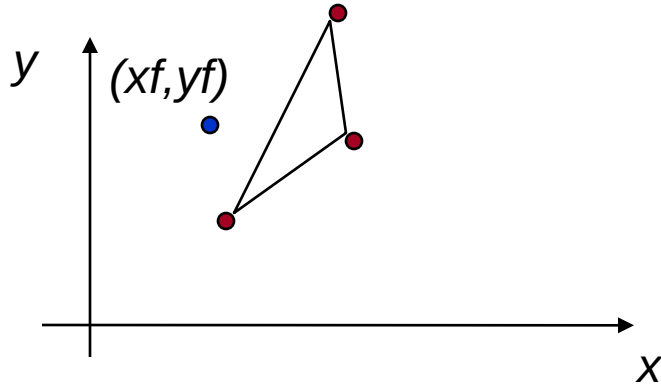
$$y' = y \cdot sy$$

$$\text{Scaling matrix : } S = \begin{pmatrix} sx & 0 \\ 0 & sy \end{pmatrix}$$

$$P' = S \cdot P$$

Scaling

Fixed point scaling



$$x' = x \cdot sx + x_f (1-sx)$$

$$y' = y \cdot sy + y_f (1-sy)$$

Rotation

Translation vector:

$$T = \begin{pmatrix} tx \\ ty \end{pmatrix} \quad P' = P + T$$

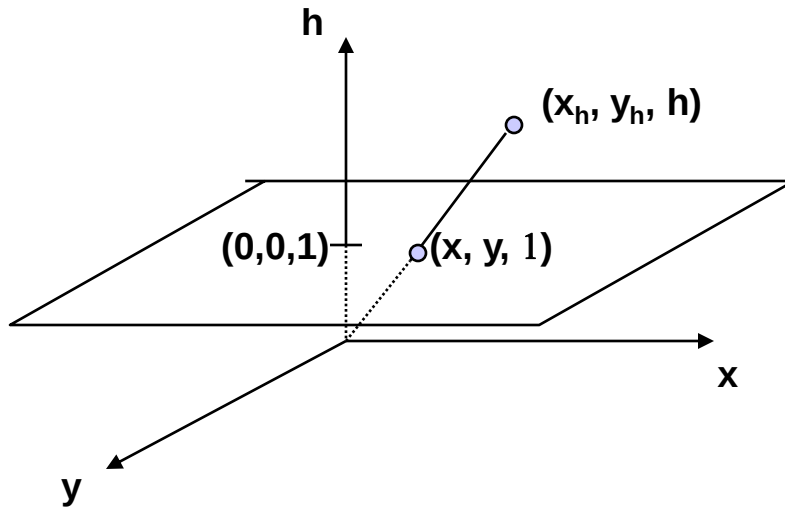
Rotation matrix:

$$R = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$
$$P' = R . P$$

Scaling matrix:

$$S = \begin{pmatrix} sx & 0 \\ 0 & sy \end{pmatrix} \quad P' = S . P$$

Homogeneous Coordinates



$$(x, y) \longrightarrow (x_h, y_h, h)$$

$$\longrightarrow (h.x, h.y, h)$$

$$\longrightarrow (x, y, 1)$$

$$x_h = h.x$$

$$y_h = h.y$$

Homogeneous Coordinates

$$(x,y) \longrightarrow (x_h, y_h, h)$$

$$x_h = h.x$$

$$y_h = h.y$$

$$(x_h, y_h, h) = (h.x, h.y, h)$$

Ex:

$(3,5)$	$\longrightarrow$	$(3, 5, 1)$	$h=1$
	$\longrightarrow$	$(6,10,2)$	$h=2$
	$\longrightarrow$	$(0.3, 0.5, 0.1)$	$h=0.1$

Translation

$$x' = x + tx$$

$$y' = y + ty$$

$$P' = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \quad P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\text{translation matrix: } T(tx, ty) = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix}$$

$$P' = T(tx, ty) \cdot P$$

Scaling

Scaling matrix : $S = \begin{pmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{pmatrix}$

$$P' = S(sx, sy) \cdot P$$

Rotation

Rotation matrix:

$$R = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$P' = R(\theta) \cdot P$$

Composite Transformations

$$P' = M3 . M2 . M1 . P$$
$$= M . P$$

M: composite matrix

$$T(t2x, t2y) . T(t1x, t1y) = T(t1x+t2x, t1y+t2y)$$

$$R(\theta2) . R(\theta1) = R(\theta1+\theta2)$$

$$S(s2x, s2y) . S(s1x, s1y) = S(s1x.s2x, s1y.s2y)$$

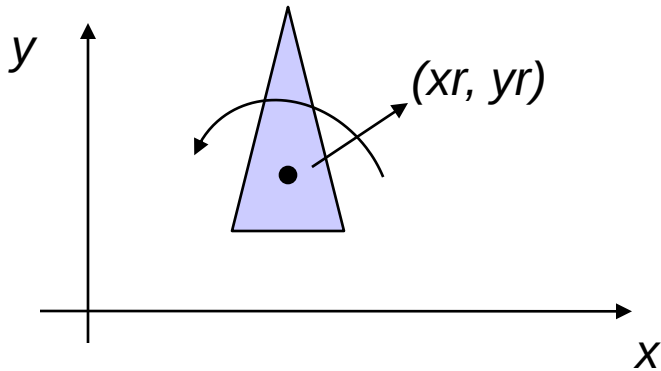
Inverse Transformations

$$T^{-1} = \begin{pmatrix} 0 & 0 & -tx \\ 0 & 0 & -ty \\ 0 & 0 & 1 \end{pmatrix}$$

$$R^{-1} = \begin{pmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$S^{-1} = \begin{pmatrix} 1/sx & 0 & 0 \\ 0 & 1/sy & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Pivot-point Rotation



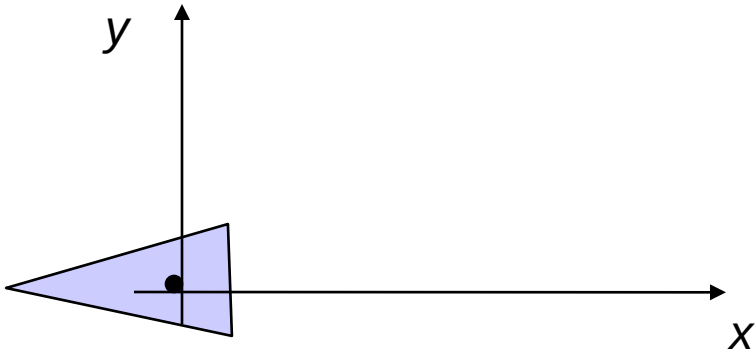
Pivot-point Rotation

1. *Translate the object so that pivot-point is moved to the origin*

$$T(-x_r, -y_r) \cdot P$$



Pivot-point Rotation



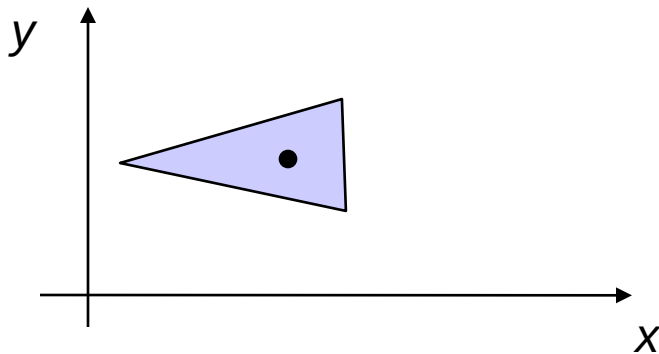
1. Translate the object so that pivot-point is moved to the origin

$$T(-x_r, -y_r) \cdot P$$

2. Rotate object about the origin

$$R(\theta) \cdot T(-x_r, -y_r) \cdot P$$

Pivot-point Rotation



1. Translate the object so that pivot-point is moved to the origin

$$T(-x_r, -y_r) \cdot P$$

2. Rotate object about the origin

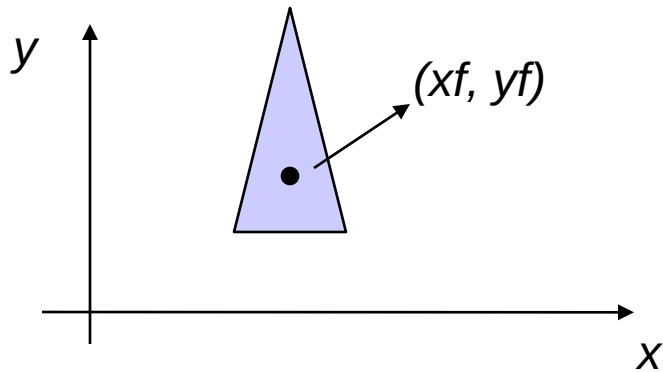
$$R(\theta) \cdot T(-x_r, -y_r) \cdot P$$

3. Translate the object so that pivot-point is returned to its original position

$$T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r) \cdot P$$

$$= R(x_r, y_r, \theta)$$

Fixed-point Scaling



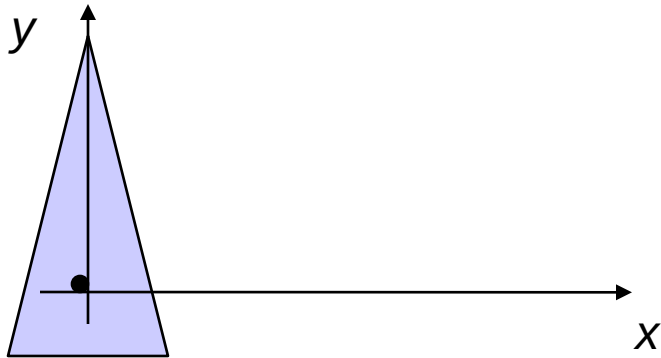
Fixed-point Scaling

1. *Translate the object so that fixed-point is moved to the origin*

$$T(-xf, -yf) \cdot P$$



Fixed-point Scaling



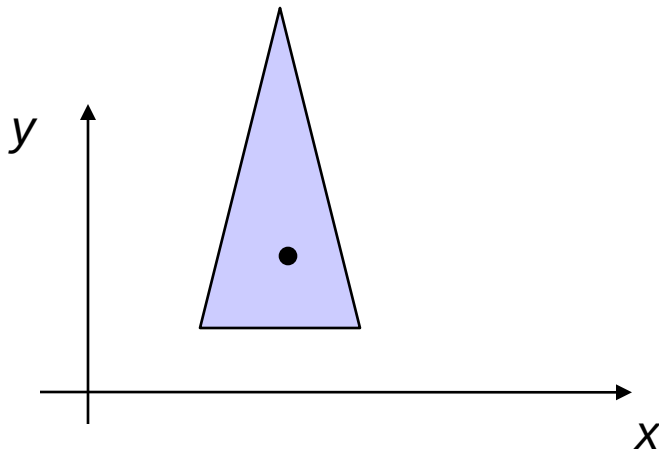
1. *Translate the object so that fixed-point is moved to the origin*

$$T(-xf, -yf) \cdot P$$

2. *Scale the object with respect to the origin*

$$S(sx, sy) \cdot T(-xf, -yf) \cdot P$$

Fixed-point Scaling



1. *Translate the object so that fixed-point is moved to the origin*

$$T(-x_f, -y_f) \cdot P$$

2. *Scale the object with respect to the origin*

$$S(s_x, s_y) \cdot T(-x_f, -y_f) \cdot P$$

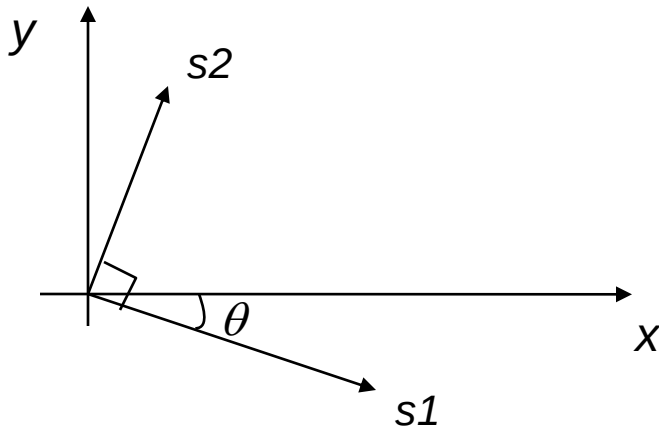
3. *Use inverse translation in step (1)*

$$T(x_f, y_f) \cdot S(s_x, s_y) \cdot T(-x_f, -y_f) \cdot P$$

$$= S(x_f, y_f, s_x, s_y)$$

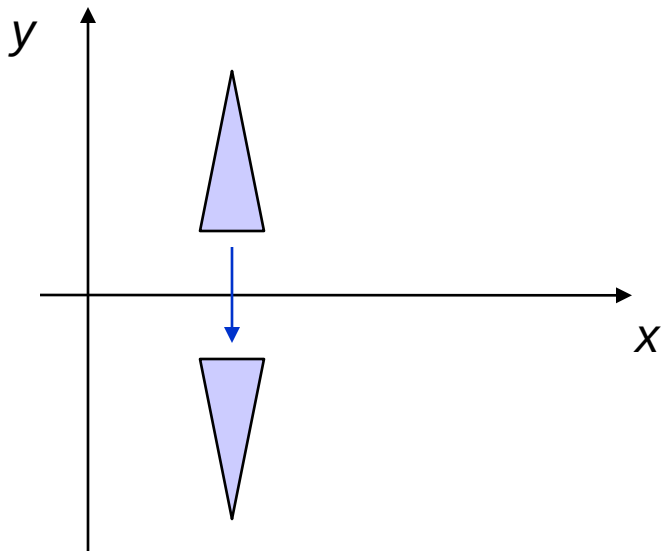
Scaling Directions

1. Rotate
2. Scale
3. Rotate back



$$R^{-1}(\theta) \cdot S(s1, s2) \cdot R(\theta)$$

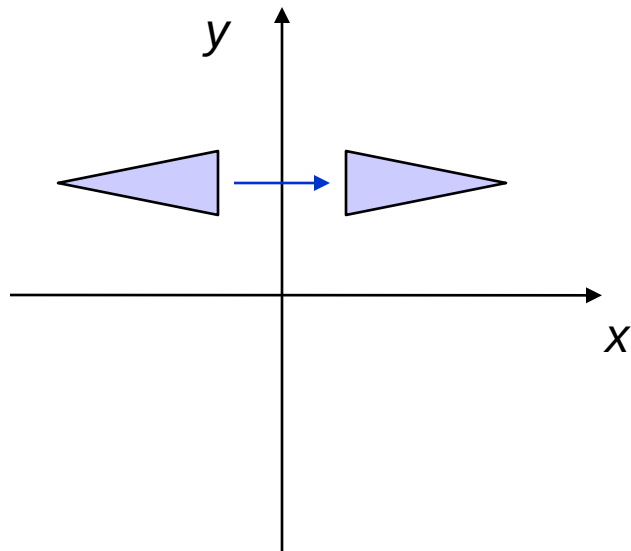
Reflection



Reflection wrt x-axis

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

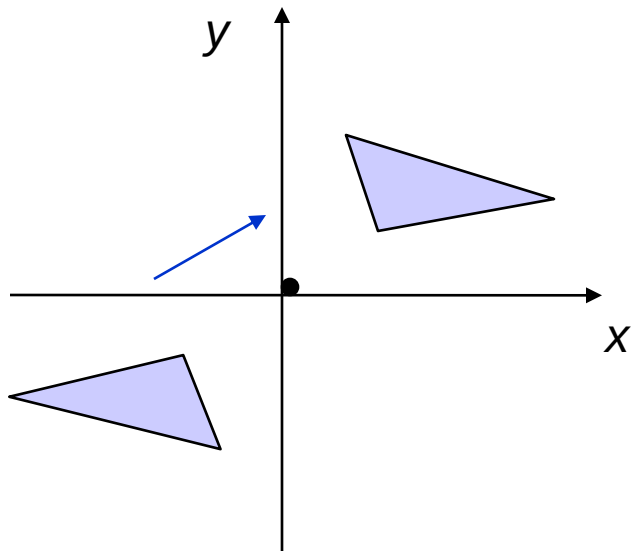
Reflection



Reflection wrt y-axis

$$\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

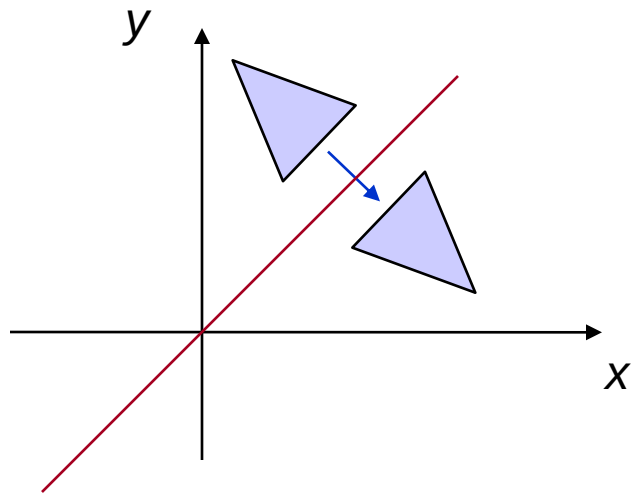
Reflection



Reflection wrt origin

$$\begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Reflection

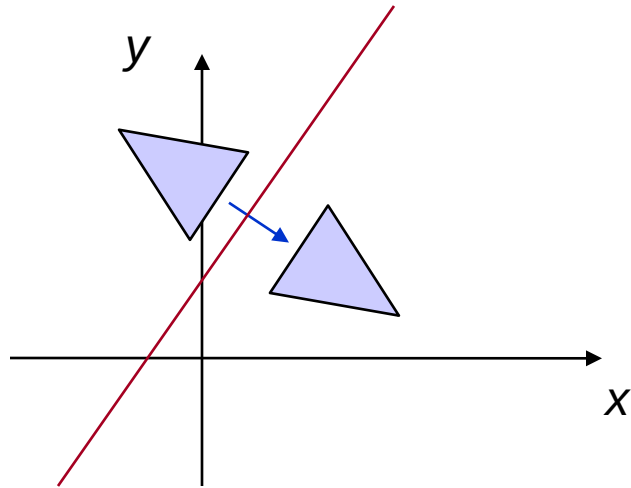


Reflection wrt the line $y=x$

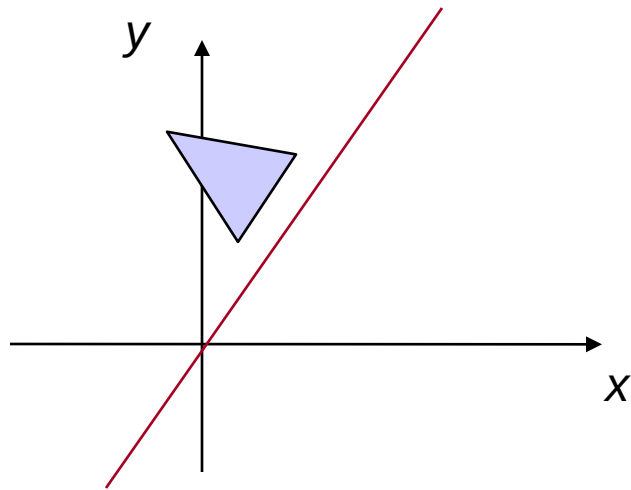
$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Reflection

Reflection wrt the line $y=mx+b$



Reflection

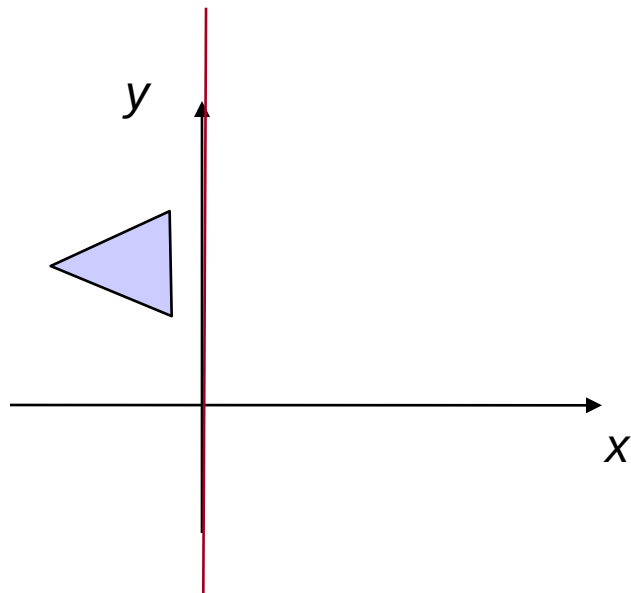


Reflection wrt the line $y=mx+b$

- 1. Translate the line so that it passes through the origin***

T

Reflection

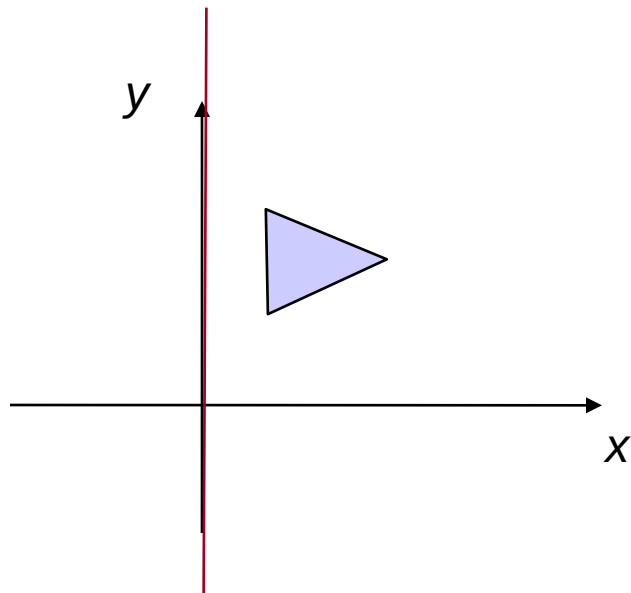


Reflection wrt the line $y=mx+b$

1. Translate the line so that it passes through the origin

2. Rotate line onto one of the axis
 $R(\theta). T$

Reflection



Reflection wrt the line $y=mx+b$

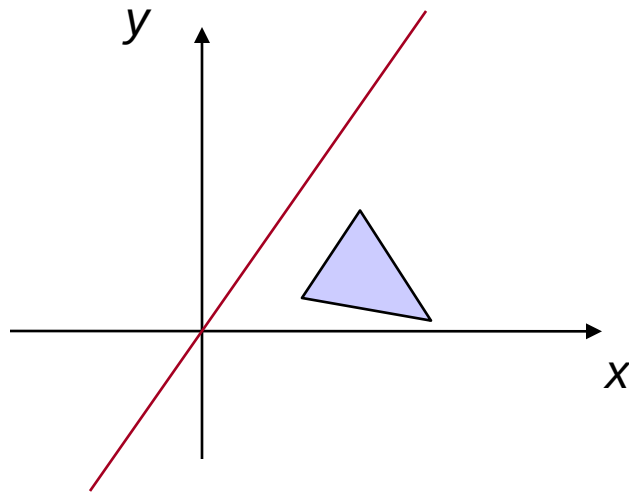
1. Translate the line so that it passes through the origin

2. Rotate line onto one of the axis

3. Reflection about the axis

$R_f . R(\theta) . T$

Reflection

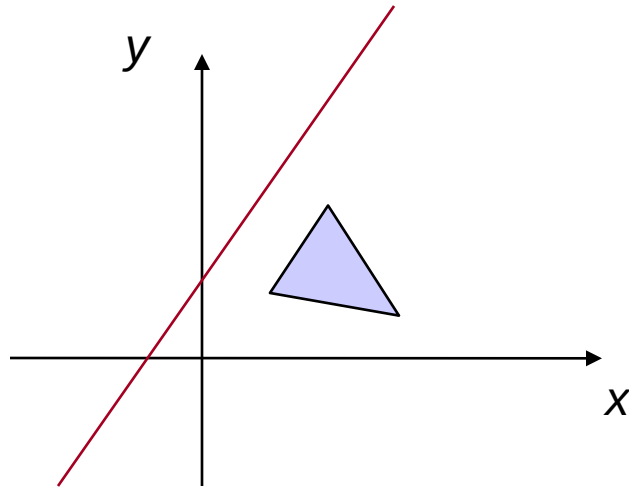


Reflection wrt the line $y=mx+b$

- 1. Translate the line so that it passes through the origin***
- 2. Rotate line onto one of the axis***
- 3. Reflection about the axis***
- 4. Translate the line to its original position***

$$R^{-1}(\theta) \cdot Rf \cdot R(\theta) \cdot T$$

Reflection

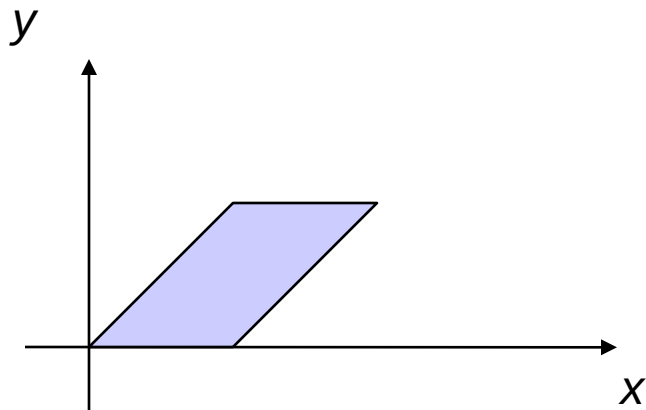
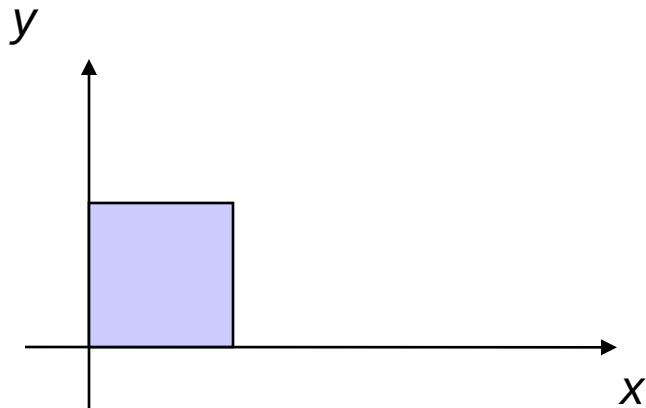


Reflection wrt the line $y=mx+b$

- 1. Translate the line so that it passes through the origin***
- 2. Rotate line onto one of the axis***
- 3. Reflection about the axis***
- 4. Translate the line to its original position***

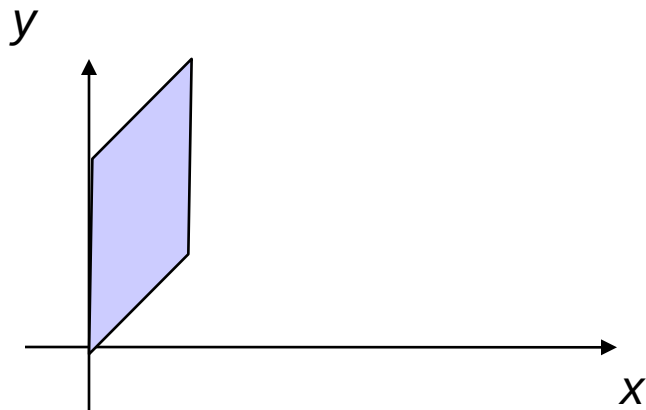
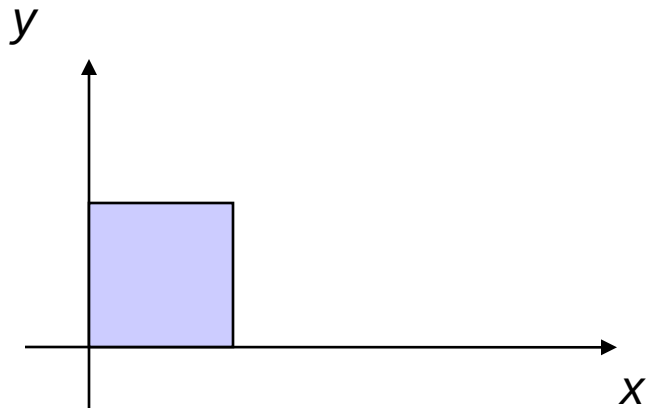
$$T^{-1} \cdot R^{-1}(\theta) \cdot Rf \cdot R(\theta) \cdot T$$

Shear



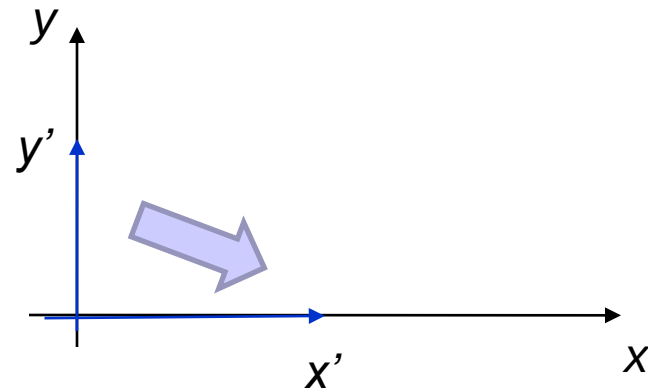
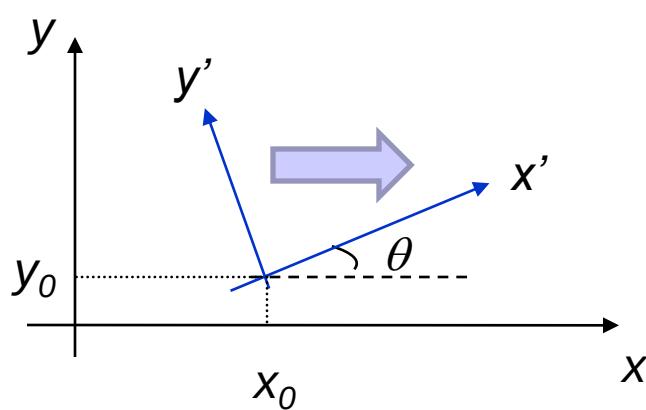
$$\begin{pmatrix} 1 & shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Shear



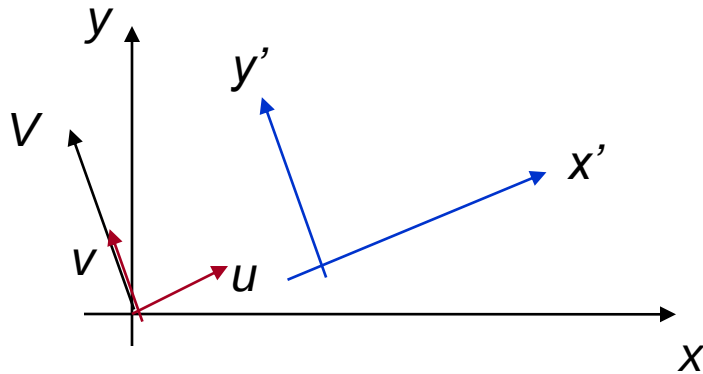
$$\begin{pmatrix} 1 & 0 & 0 \\ \text{shy} & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Transformations between 2D Coordinate Systems



$$M = R(-\theta) \cdot T(-x_0, -y_0)$$

Transformations between 2D Coordinate Systems



$$\mathbf{v} = (v_x, v_y)$$

$$\mathbf{u} = (v_y, -v_x)$$

$$\mathbf{R} = \begin{pmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

OpenGL

- **glTranslate* (tx, ty, tz)**
 - f (float)
 - d (double)
- **glRotate* (theta, vx, vy, vz)**
 - theta: rotation angle in degrees
 - (vx, vy, vz) vector defines the orientation of the rotation axis that passes through the coordinate origin. (0,0,1) in 2D
- **glScale* (sx, sy, sz)**

```

void rectangles1 (void) {
    glMatrixMode (GL_MODELVIEW);          // set modelview matrix
    glLoadIdentity();
    ...
    glColor3f (1.0, 0.0, 0.0);           // red
    glRecti(0,0,80,40);
    glColor3f (0.0, 1.0, 0.0);           // green
    glTranslatef(10.0, 10.0, 0.0);
    glRecti(0,0,80,40);
    glColor3f (0.0, 0.0, 1.0);           // blue
    glTranslatef(10.0, 10.0, 0.0);
    glRecti(0,0,80,40);
    glFlush ( );
}

void main (int argc, char** argv) {
    ...
    glutDisplayFunc (rectangles1);
    ...
    glutMainLoop ( );
}

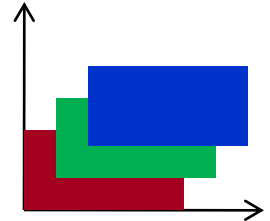
```

matrix

I

I.T

I.T.T

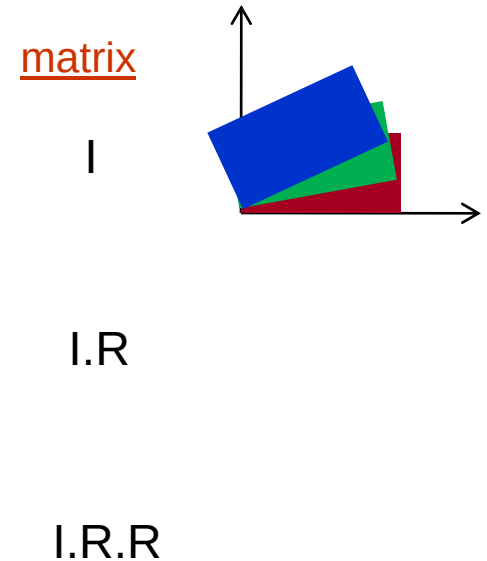


```

void rectangles2 (void) {
    glMatrixMode (GL_MODELVIEW);          // set modelview matrix
    glLoadIdentity();
    ...
    glColor3f (1.0, 0.0, 0.0);           // red
    glRecti(0,0,80,40);
    glColor3f (0.0, 1.0, 0.0);           // green
    glRotatef(20.0, 0.0, 0.0, 0.0);
    glRecti(0,0,80,40);
    glColor3f (0.0, 0.0, 1.0);           // blue
    glRotatef(20.0, 0.0, 0.0, 0.0);
    glRecti(0,0,80,40);
    glFlush ( );
}

void main (int argc, char** argv) {
    ...
    glutDisplayFunc (rectangles2);
    ...
    glutMainLoop ( );
}

```

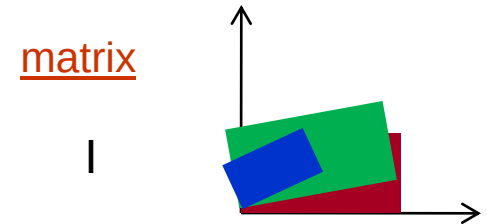


```

void rectangles3 (void) {
    glMatrixMode (GL_MODELVIEW);          // set modelview matrix
    glLoadIdentity();
    ...
    glColor3f (1.0, 0.0, 0.0);           // red
    glRecti(0,0,80,40);
    glColor3f (0.0, 1.0, 0.0);           // green
    glRotatef(20.0, 0.0, 0.0, 0.0);
    glRecti(0,0,80,40);
    glColor3f (0.0, 0.0, 1.0);           // blue
    glRotatef(20.0, 0.0, 0.0, 0.0);
    glScalef(0.5, 0.5, 1.0);
    glRecti(0,0,80,40);
    glFlush ( );
}

void main (int argc, char** argv) {
    ...
    glutDisplayFunc (rectangles3);
    ...
    glutMainLoop ( );
}

```



I

I.R

I.R.R.S

OpenGL

```
glMatrixMode(GL_MODELVIEW);  
glLoadIdentity ( );  
glMultMatrixf(M2);  
glMultMatrixf(M1);  
  
/* M = M2 . M1 */
```


OpenGL

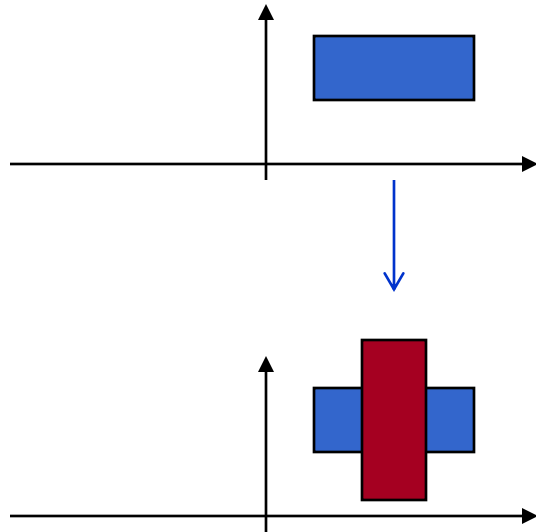
- **glMatrixMode(GL_MODELVIEW)**
 - sets up the matrix for transformations (4x4 modelview matrix)
- **glLoadIdentity ()**
 - assigns identity matrix to the current matrix
- **glLoadMatrix*(16-element array)**
 - assigns a 16-element array (in column major order) to the current matrix
- **glMultMatrix*(16-element array)**
 - postmultiplies a 16-element array (M') with the current matrix (M) : $M \leftarrow M.M'$

OpenGL

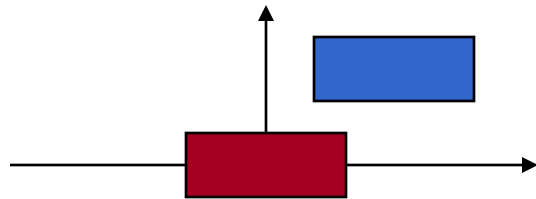
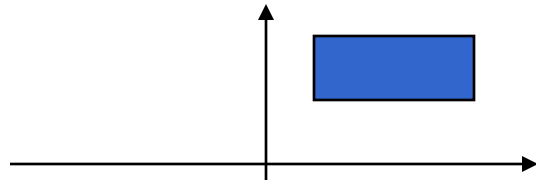
Matrix Stack

- Top matrix on the stack is the “current matrix”
- Initially stack contains identity matrix
- Maximum stack depth is 32
- **glGetIntegerv (GL_MAX_MODELVIEW_STACK_DEPTH, stackSize)**
 - returns the number of positions available in the modelview stack
- **glGetIntegerv (GL_MODELVIEW_STACK_DEPTH, numMats)**
 - returns the number of matrices currently in the stack
- **glPushMatrix()**
 - copies the current matrix at the top of the stack
- **glPopMatrix()**
 - destroys the matrix at the top of the stack

OpenGL



OpenGL



```
glMatrixMode(GL_MODELVIEW);
```

```
glColor3f(0.0, 0.0, 1.0);
```

```
Recti(50, 100, 200, 150);
```

```
glColor3f(1.0, 0.0, 0.0);
```

```
glTranslatef(125.0, 125.0, 0.0);
```

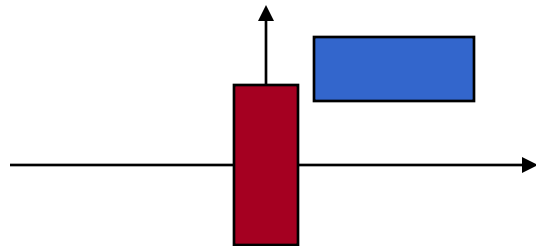
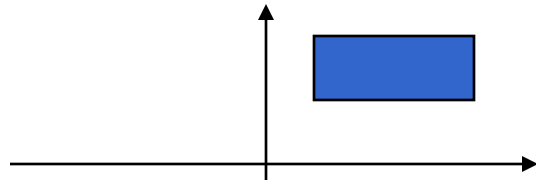
```
glRotatef(90.0, 0.0, 0.0, 1.0);
```

```
glTranslatef(-125.0, -125.0, 0.0);
```

```
Recti(50, 100, 200, 150);
```

```
/* T(125,125).R(90).T(-125,-125).P */
```

OpenGL



```
glMatrixMode(GL_MODELVIEW);
```

```
glColor3f(0.0, 0.0, 1.0);
```

```
Rect(50, 100, 200, 150);
```

```
glColor3f(1.0, 0.0, 0.0);
```

```
glTranslatef(125.0, 125.0, 0.0);
```

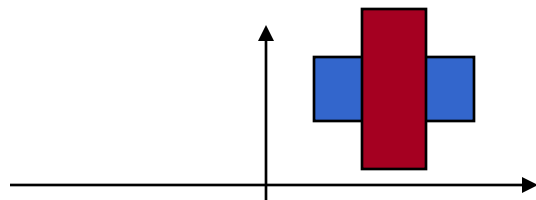
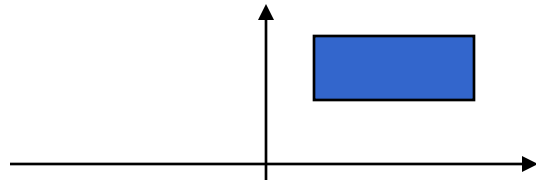
```
glRotatef(90.0, 0.0, 0.0, 1.0);
```

```
glTranslatef(-125.0, -125.0, 0.0);
```

```
Rect(50, 100, 200, 150);
```

```
/* T(125,125).R(90).T(-125,-125).P */
```

OpenGL



```
glMatrixMode(GL_MODELVIEW);
```

```
glColor3f(0.0, 0.0, 1.0);
```

```
Recti(50, 100, 200, 150);
```

```
glColor3f(1.0, 0.0, 0.0);
```

```
glTranslatef(125.0, 125.0, 0.0);
```

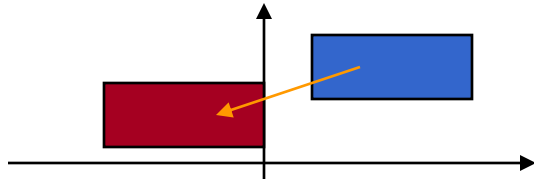
```
glRotatef(90.0, 0.0, 0.0, 1.0);
```

```
glTranslatef(-125.0, -125.0, 0.0);
```

```
Recti(50, 100, 200, 150);
```

```
/* T(125,125).R(90).T(-125,-125).P */
```

OpenGL



```
glMatrixMode(GL_MODELVIEW);
```

```
glColor3f(0.0, 0.0, 1.0);
```

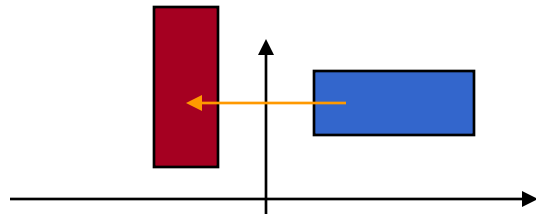
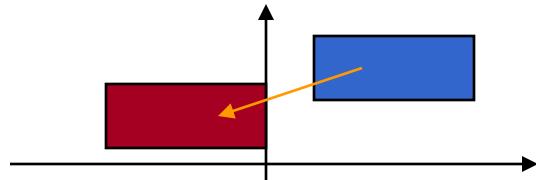
```
Rect(50, 100, 200, 150);
```

```
glColor3f(1.0, 0.0, 0.0);
```

```
glTranslatef(-200.0, -50.0, 0.0);
```

```
Rect(50, 100, 200, 150);
```

OpenGL



```
glMatrixMode(GL_MODELVIEW);
```

```
glColor3f(0.0, 0.0, 1.0);
```

```
Recti(50, 100, 200, 150);
```

```
glColor3f(1.0, 0.0, 0.0);
```

```
glTranslatef(-200.0, -50.0, 0.0);
```

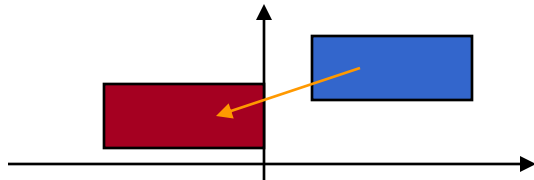
```
Recti(50, 100, 200, 150);
```

```
glLoadIdentity ( );
```

```
glRotatef(90.0, 0.0, 0.0, 1.0);
```

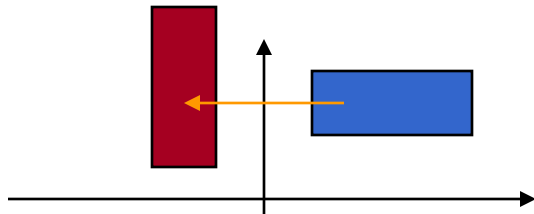
```
Recti(50, 100, 200, 150);
```


OpenGL



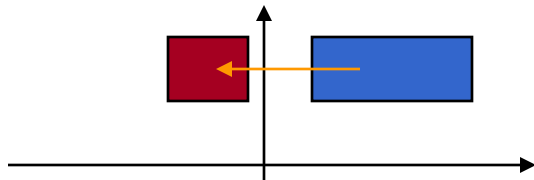
```
glMatrixMode(GL_MODELVIEW);
```

```
glColor3f(0.0, 0.0, 1.0);  
Recti(50, 100, 200, 150);
```



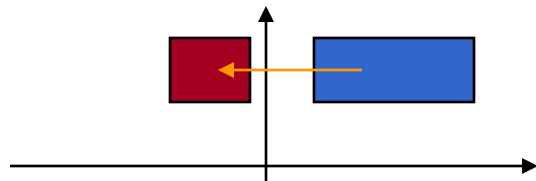
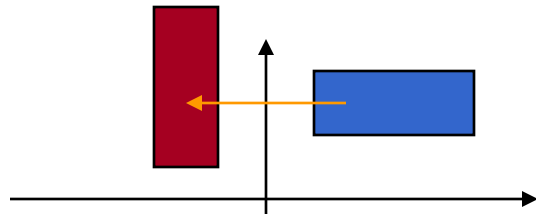
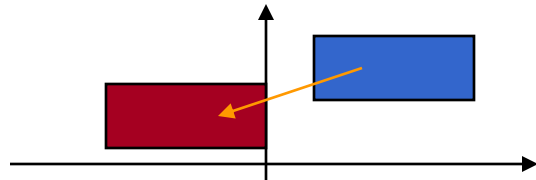
```
glColor3f(1.0, 0.0, 0.0);  
glTranslatef(-200.0, -50.0, 0.0);  
Recti(50, 100, 200, 150);
```

```
glLoadIdentity ( );  
glRotatef(90.0, 0.0, 0.0, 1.0);  
Recti(50, 100, 200, 150);
```



```
glLoadIdentity ( );  
glScalef(-0.5, 1.0, 1.0);  
Recti(50, 100, 200, 150);
```

OpenGL



```
glMatrixMode(GL_MODELVIEW);
```

```
glColor3f(0.0, 0.0, 1.0);  
Rectf(50, 100, 200, 150);
```

```
glPushMatrix();  
glColor3f(1.0, 0.0, 0.0);  
glTranslatef(-200.0, -50.0, 0.0);  
Rectf(50, 100, 200, 150);
```

```
glPopMatrix();  
glPushMatrix();
```

```
glRotatef(90.0, 0.0, 0.0, 1.0);  
Rectf(50, 100, 200, 150);
```

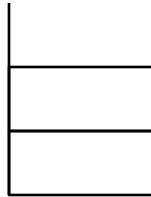
```
glPopMatrix();  
glScalef(-0.5, 1.0, 1.0);  
Rectf(50, 100, 200, 150);
```

OpenGL

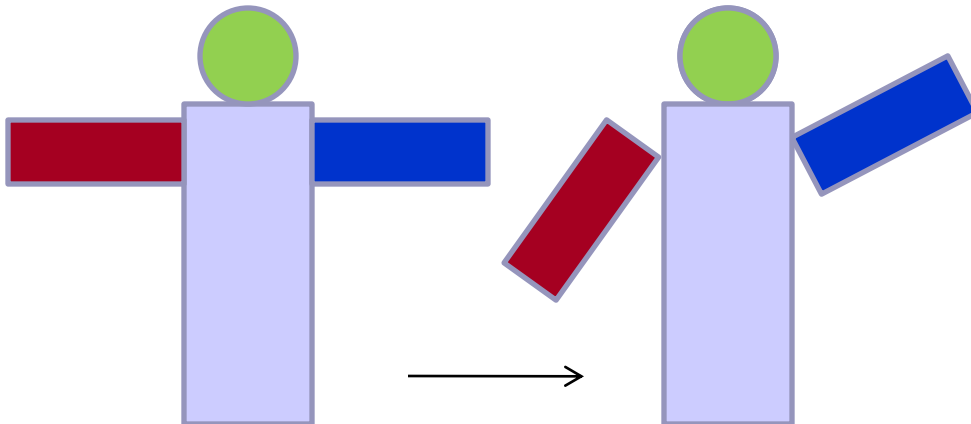
matrix

I

stack



.

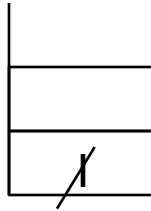


OpenGL

matrix

I.T

stack

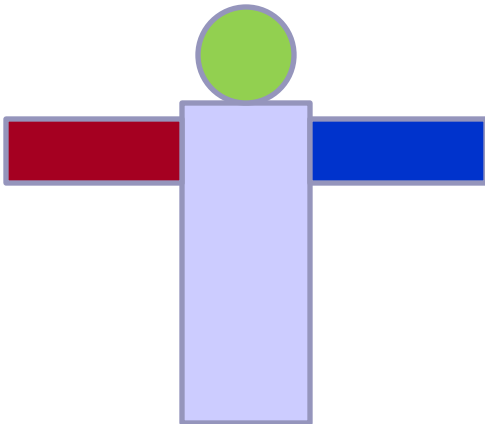


```
glPushMatrix();
```

```
    translate_head();
```

```
    draw_head();
```

```
glPopMatrix();
```

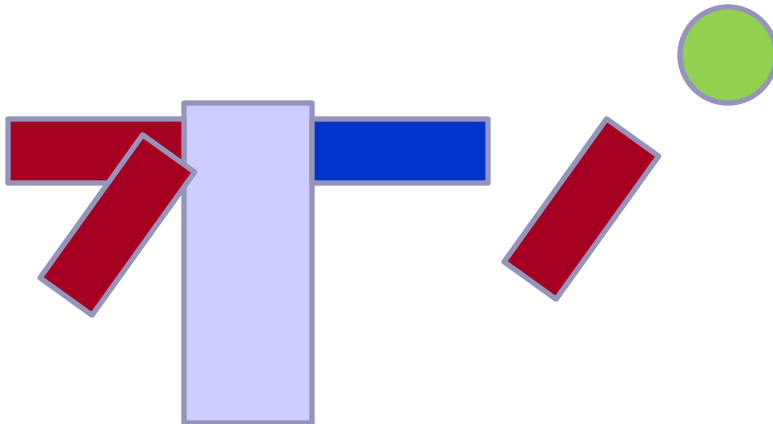
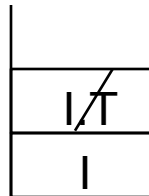


OpenGL

matrix

I .T.R

stack



```
glPushMatrix();
```

```
    translate_head();
```

```
    draw_head();
```

```
glPopMatrix();
```

```
glPushMatrix();
```

```
    translate_body();
```

```
glPushMatrix();
```

```
    rotate_left_arm();
```

```
    draw_left_arm();
```

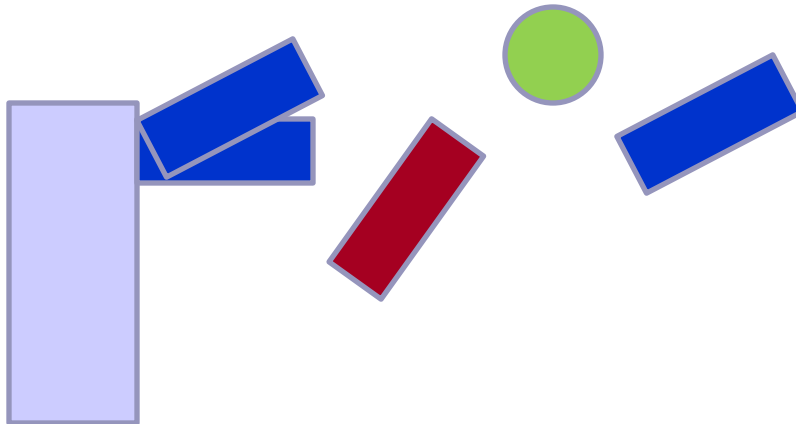
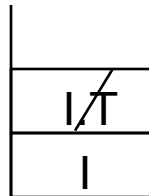
```
glPopMatrix();
```

OpenGL

matrix

I .T.R

stack



```
glPushMatrix();
```

```
    translate_head();
```

```
    draw_head();
```

```
glPopMatrix();
```

```
glPushMatrix();
```

```
    translate_body();
```

```
glPushMatrix();
```

```
    rotate_left_arm();
```

```
    draw_left_arm();
```

```
glPopMatrix();
```

```
glPushMatrix();
```

```
    rotate_right_arm();
```

```
    draw_right_arm();
```

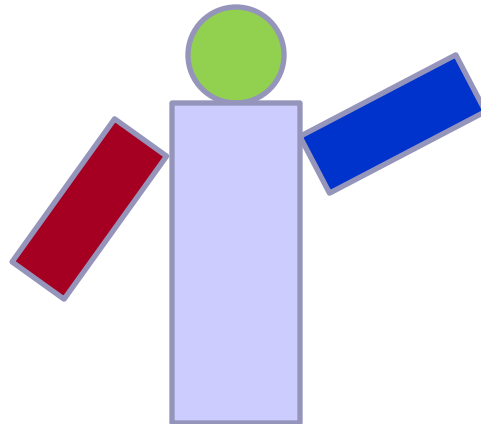
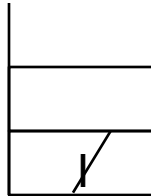
```
glPopMatrix();
```

OpenGL

matrix

I.T

stack



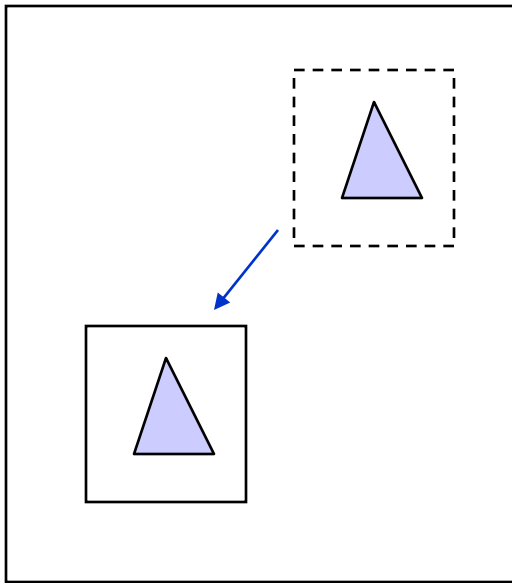
```
glPushMatrix();  
    translate_head();  
    draw_head();  
glPopMatrix();  
glPushMatrix();  
    translate_body();  
glPushMatrix();  
    rotate_left_arm();  
    draw_left_arm();  
glPopMatrix();  
glPushMatrix();  
    rotate_right_arm();  
    draw_right_arm();  
glPopMatrix();  
    draw_body();  
glPopMatrix();
```

```
void keybrd(GLubyte key, GLint xmouse, GLint ymouse) {

    switch (key){
    case 'r':    { glColor3f (1.0, 0.0, 0.0);
                  break;
                }
    case 'g':    { glColor3f (0.0, 1.0, 0.0);
                  break;
                }
    case 'b':    { glColor3f (0.0, 0.0, 1.0);
                  break;
                }
    default: break;
    }
}

void main (int argc, char** argv) {
    ...
    glutKeyboardFunc(keybrd);
    ...
    glutMainLoop ( );
}
```


Raster Methods

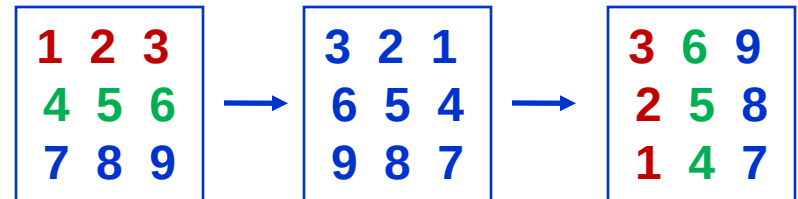


Translation

***Block transfer of the refresh
buffer area***

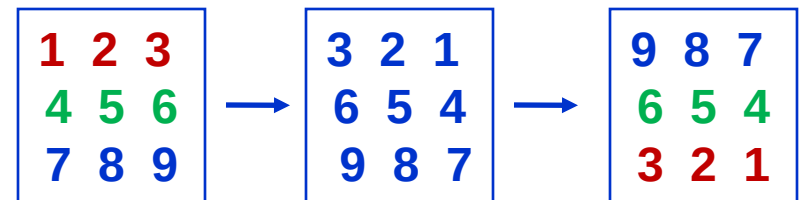
Raster Methods

Rotation



90° counterclockwise:

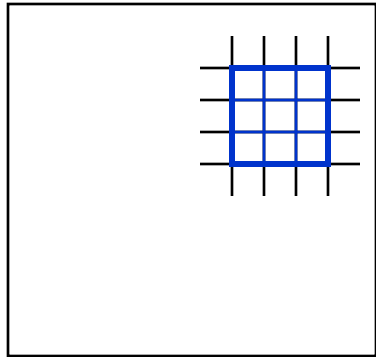
- 1. reverse pixel values in each row*
- 2. interchange rows and columns*



180°:

- 1. reverse the order of elements in each row*
- 2. reverse the order of rows*

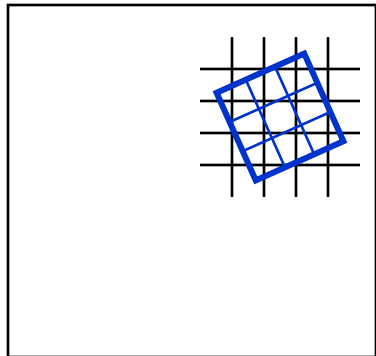
Raster Methods



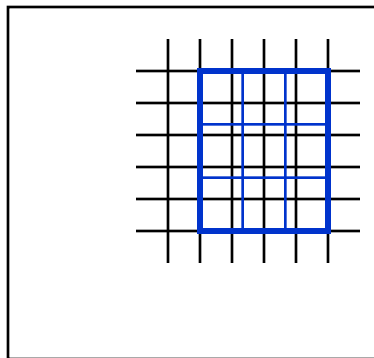
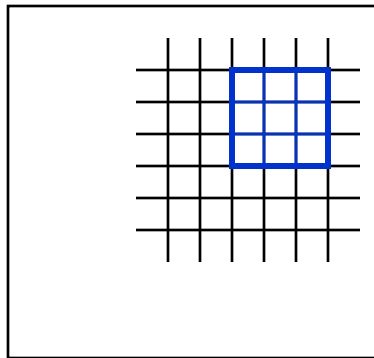
Rotation

Other cases:

- 1. calculate the amount of overlap with the rotated pixel areas*
- 2. compute the color by averaging the colors of the overlapped source pixels, weighted by their percentage of area overlap.*



Raster Methods

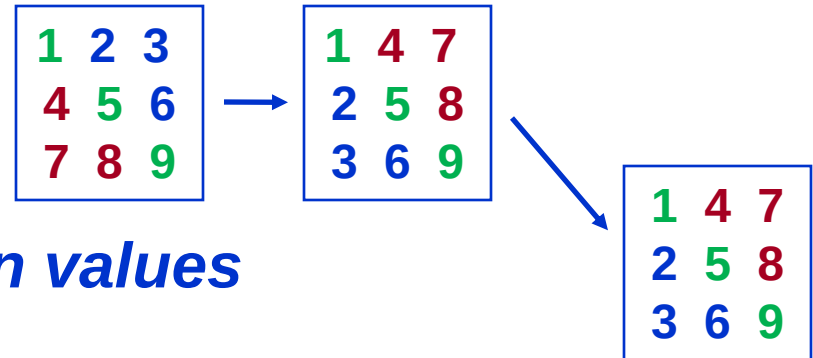


Scaling

- 1. scale pixel areas in the original block using s_x , s_y*
- 2. assign color of the destination pixel according to the overlap area.*

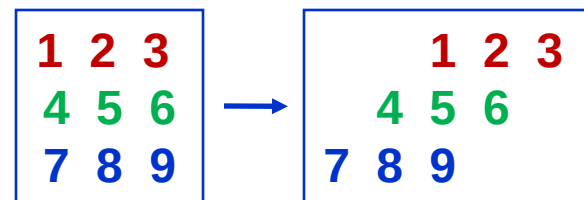
Raster Methods

Reflection



- 1. reverse row and column values*
- 2. translate*

Shear



*Shift the position of array values along rows
or columns*